Team members name:	1. Lê Đức Anh 2. Ông Phước Đại 3. Đặng Quốc Khang
Local Students ID:	1. GCH220282 2. GCS220457 3. GCS220468
Academic Year:	2025-2026
Module:	AMD201- Advanced Microservices Development and Deployment
Submission Date:	24/03/2026
Due Date	24/03/2026

Link Github: https://github.com/AnhLe-dev/AMD201

Table of Contents

1.Introduction 3
1.1 Introduction to your group 3
1.2 Introduction to your project 3
2. URLS Shortener system 3
2.1 Overview systems URL 3
2.2 System Functionality URL 4
2.3 Microsoft SQL database 15
3.Conclusion 16
4. Self-evaluation 16
4.1 Contribution 16
4.2 Challenges 18
4.3 Learning Outcomes 18
Student Assessment Submission and Declaration 19
Plagiarism 19

Figure 1. Home Page 5
Figure 2. Register Page 6
Figure 3. Login Page 7
Figure 4. User Home 8
Figure 5. Create Page 9
Figure 6. My Links Page 10
Figure 7. QR Code 11
Figure 8. Short URL Detail Page 11
Figure 9. Edit Short URL Page 12
Figure 10. Detele Short URL Page 12
Figure 11. Admin Page 13
Figure 12. Incorrect information entered 14
Figure 13. Incorrect URL entered 14
Figure 14. Data of URL links in SQL 15
Figure 15. Data of Users in System 16

1.Introduction

1.1 Introduction to your group

Our group, Group 3, consists of Ông Phước Đại, Đặng Quốc Khang, and Lê Đức Anh. Ông Phước Đại and Đặng Quốc Khang are responsible for proposing ideas, designing the system architecture, and ensuring that the code functions correctly. Meanwhile, Lê Đức Anh is in charge of system testing, reviewing the code, preparing the project report, and developing the presentation script.

1.2 Introduction to your project

In today's digital landscape, sharing long, cluttered links can be impractical and often degrades the user experience. URL shorteners like Bitly and TinyURL have become essential tools by transforming these complex addresses into manageable, unique codes. This project focuses on designing and implementing a similar high-performance service using .NET Core, moving beyond basic functionality to explore the core principles of scalable API design.A primary objective of this assignment is the integration of modern DevOps practices. We aim to build a fully automated CI/CD pipeline leveraging GitHub Actions and Docker to ensure a seamless transition from code commit to cloud deployment on platforms like Render. By incorporating features such as distributed caching with Redis and exploring Microservices architecture, this project demonstrates a comprehensive approach to building reliable, cloud-native applications in a professional engineering environment.

2. URLS Shortener system

2.1 Overview systems URL

The system is a web-based URL shortening application developed using the ASP.NET MVC framework, designed to transform long URLs into shorter, more manageable links for improved usability and sharing efficiency. In addition to standard shortened links, the system supports QR code generation, enabling quick access via mobile devices. A notable feature is the ability for users to create custom aliases for their shortened URLs, enhancing personalization and flexibility. The application also integrates authentication and role-based authorization, allowing users to register and log in securely. Administrators have full access to all user-generated links, while standard users are restricted to viewing and managing only their own data, ensuring privacy and data isolation. Functionally, the system supports full CRUD operations, allowing users to create, view, update, and delete their links as needed. The application is complemented by a modern, intuitive user interface that ensures ease of use and effective interaction. Overall, the system delivers a secure, flexible, and user-friendly solution aligned with modern web application practices.

2.2 System Functionality URL

In **Figure 1**,**Figure 1** the homepage is structured with a modern and user-oriented layout, beginning with a prominent hero section that introduces the core purpose of the system, which is URL shortening, accompanied by clear call-to-action buttons for registration and login. This is followed by a feature section that presents key functionalities including fast link generation, click analytics, custom aliases, QR code creation, security, and link management. A dedicated “How It Works” section then illustrates the main usage flow from account creation to link sharing and tracking, helping users quickly understand the system process. The page also includes a call-to-action area to encourage user engagement, while the footer provides essential navigation links and social connection options, ensuring accessibility and completeness of the interface.

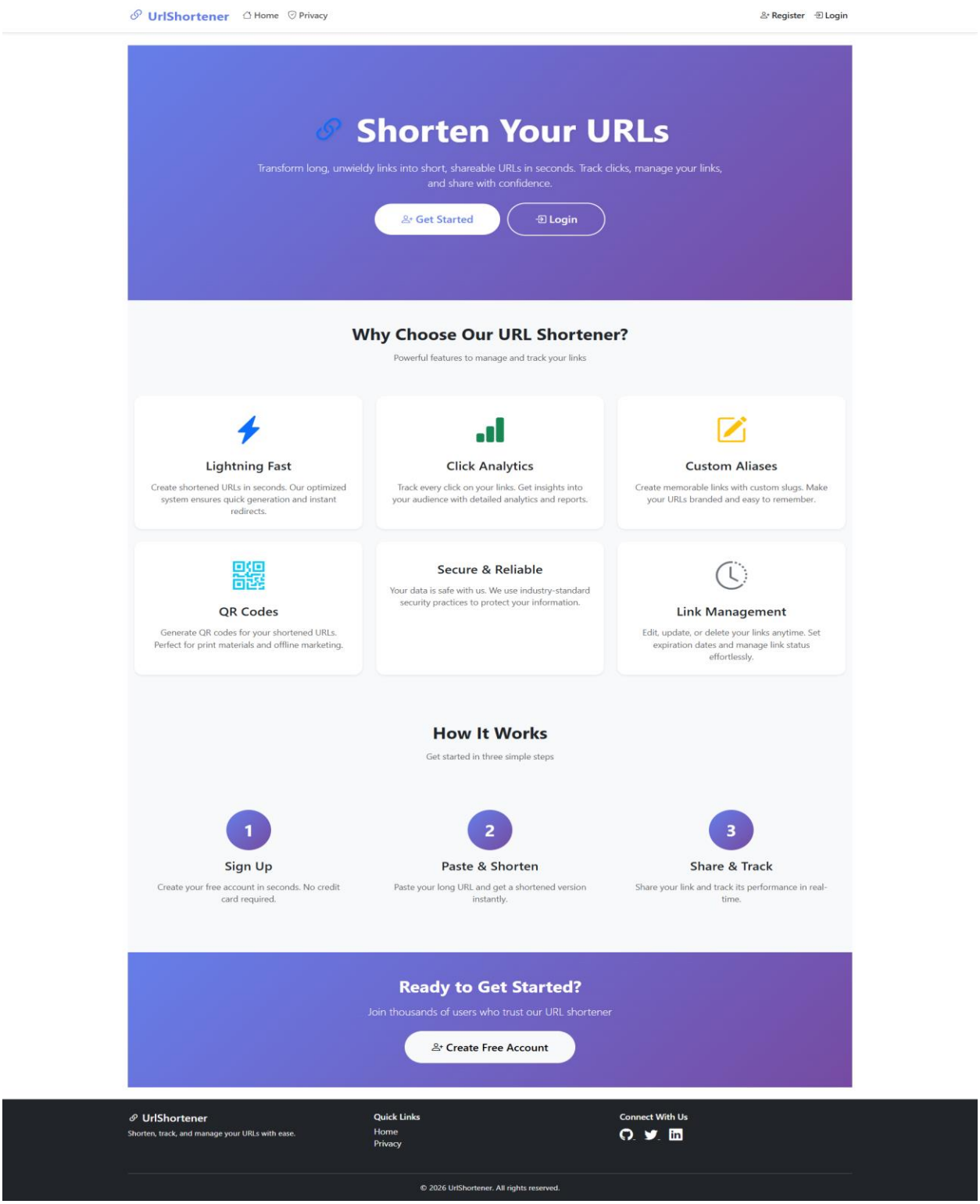

Figure 1. Home Page

In **Figure 2** the register page is designed with a clean and structured layout, centered around a user-friendly form that facilitates account creation. It includes essential input fields such as email, full name, password, and password confirmation, each supported by validation mechanisms to ensure data accuracy and security, including password strength requirements and error notifications. A clearly highlighted registration button enables users to submit their information efficiently, while an additional navigation option is provided for users who already have an account to access the login page.

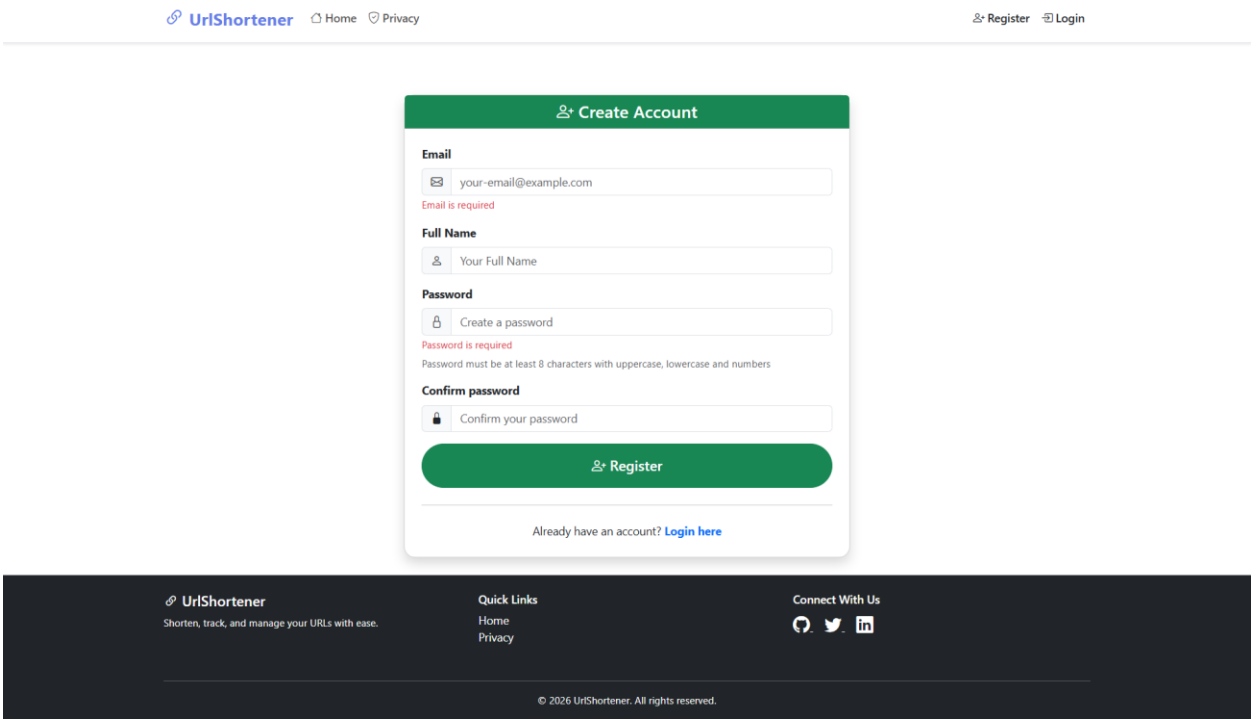

Figure 2. Register Page

In ***Figure 3***, the login page allows users to access the system by entering their registered email and password, providing a simple and efficient authentication process.

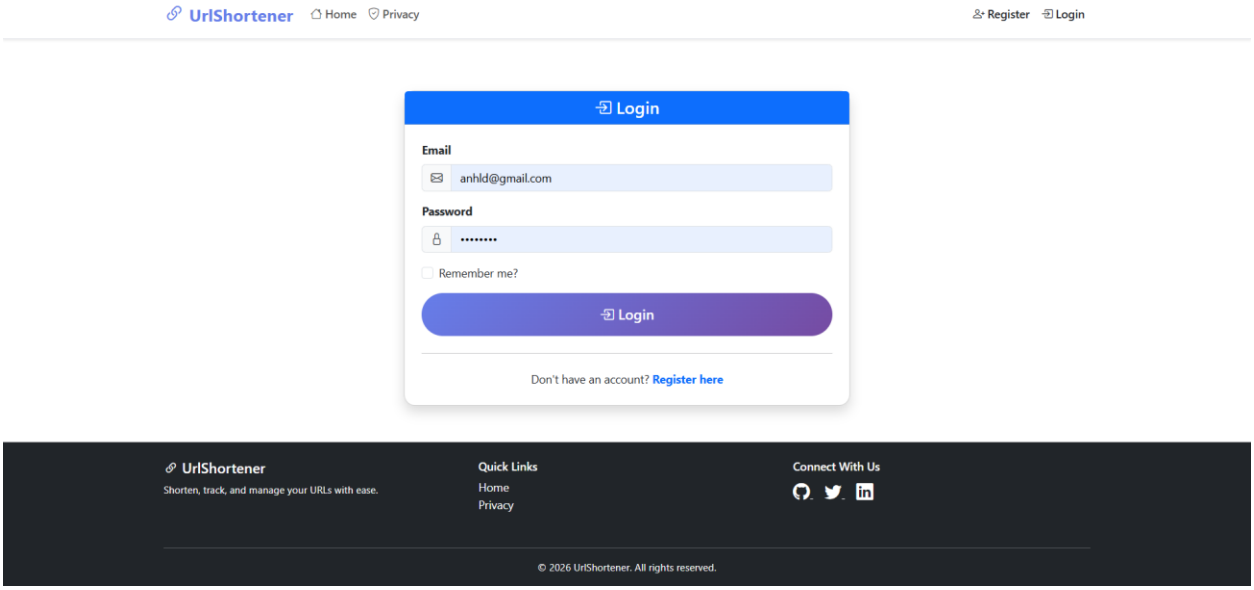

Figure 3. Login Page

In ***Figure 4***, the user homepage presents an interface similar to the Home page in **Figure 1**, with additional navigation options including “My Links”, “Create”, and “Logout” to support user-specific actions and account management.

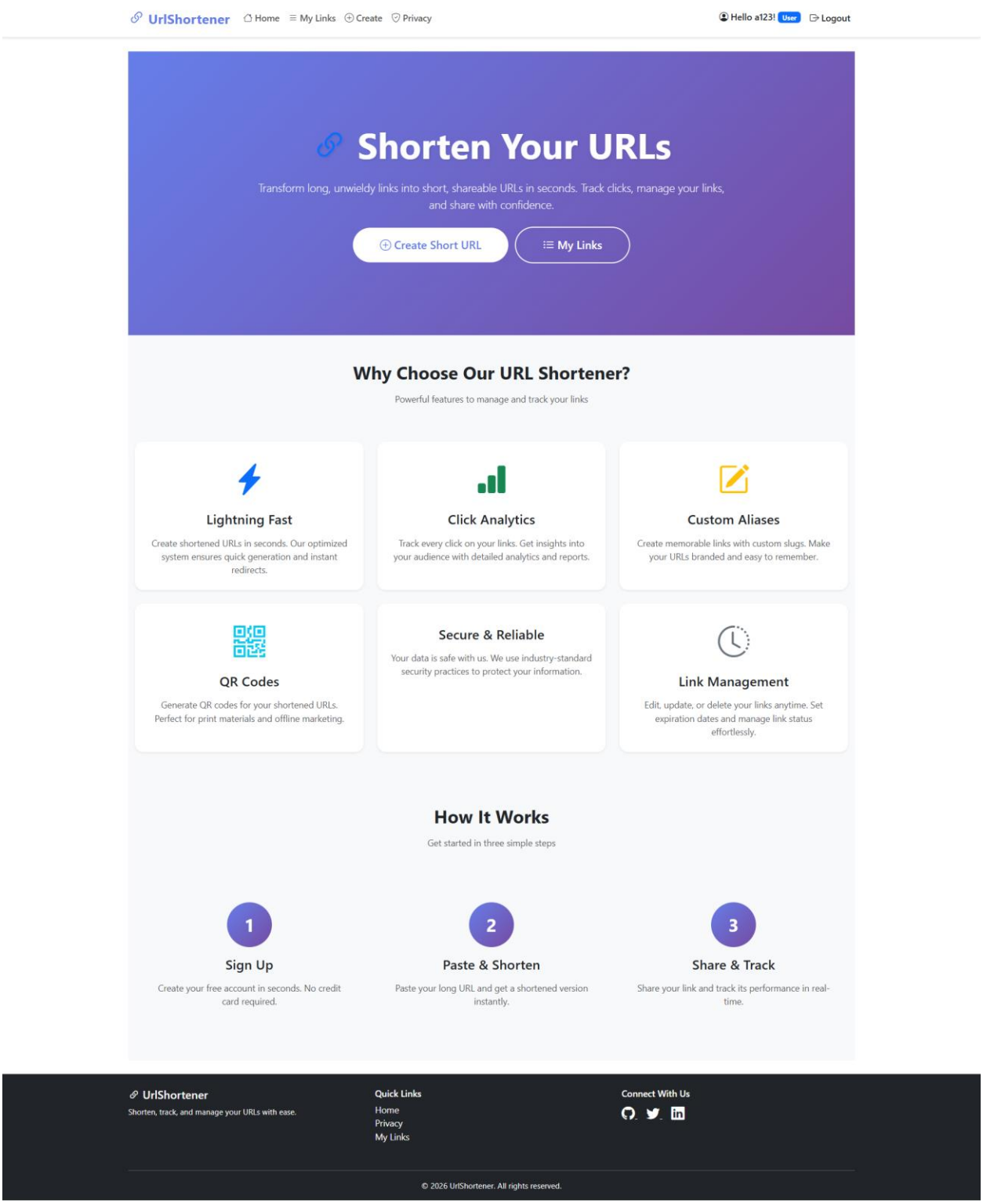

Figure 4. User Home

In ***Figure 5***, the Create page provides a structured interface for generating shortened URLs, featuring input fields for the original URL, optional custom alias, title, description, and expiration date, allowing flexible link customization. It includes clear guidance for auto-generating aliases and setting optional parameters, ensuring usability and accuracy. A primary action button enables users to create the short URL efficiently, while a secondary option allows navigation back to the link list.

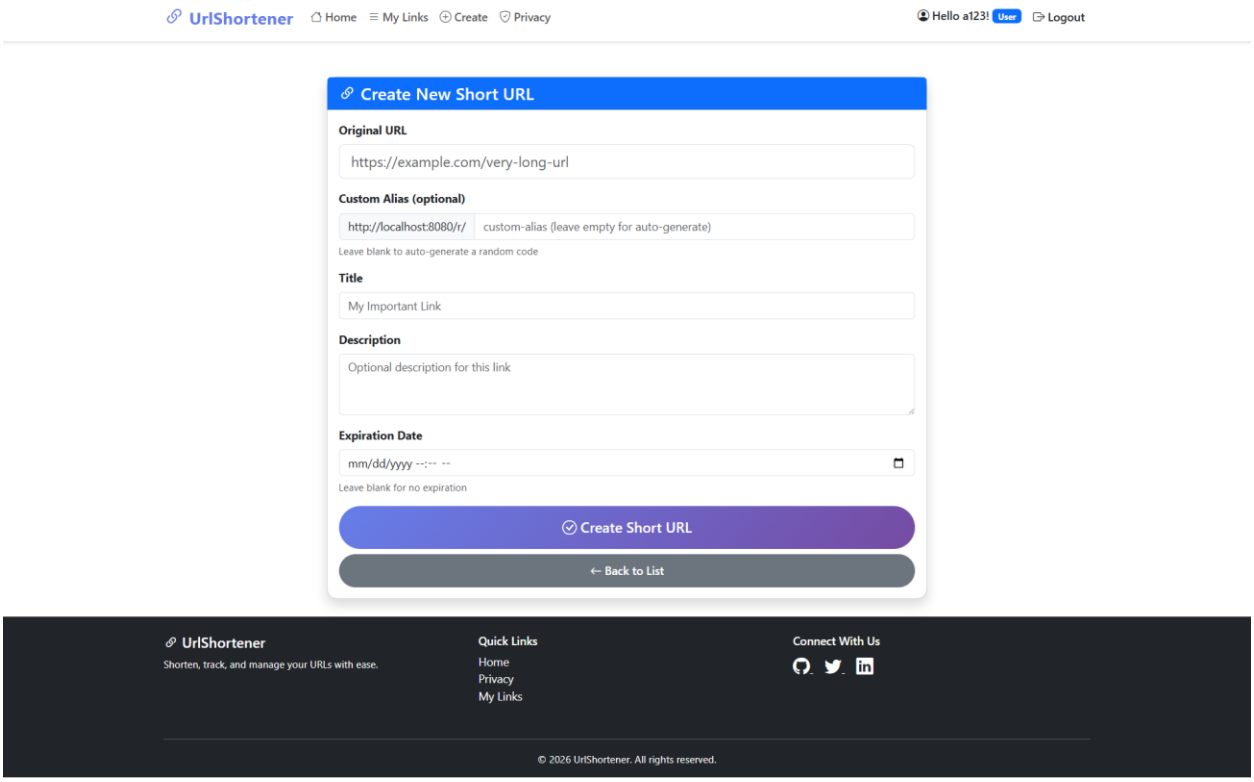

Figure 5. Create Page

In ***Figure 6***, the My Links interface is designed with a hierarchical structure that optimizes the link management experience within the administrative workflow. At the center, the system integrates a Short URL Generator tool that allows quick creation of shortened links with QR codes, serving as the primary data input. Below this, a detailed statistics table of the user's shortened URLs is presented, where monitoring and evaluation functions are performed. Key components such as Short Code, Original URL, click counts, and status are organized into columns, enabling users to easily retrieve and assess the performance of each link. Additionally, users can view QR codes, access link details, edit existing links, and delete them when necessary. The page logic is supported by an intuitive navigation system, allowing direct interaction with link attributes and ensuring a consistent and streamlined management process.

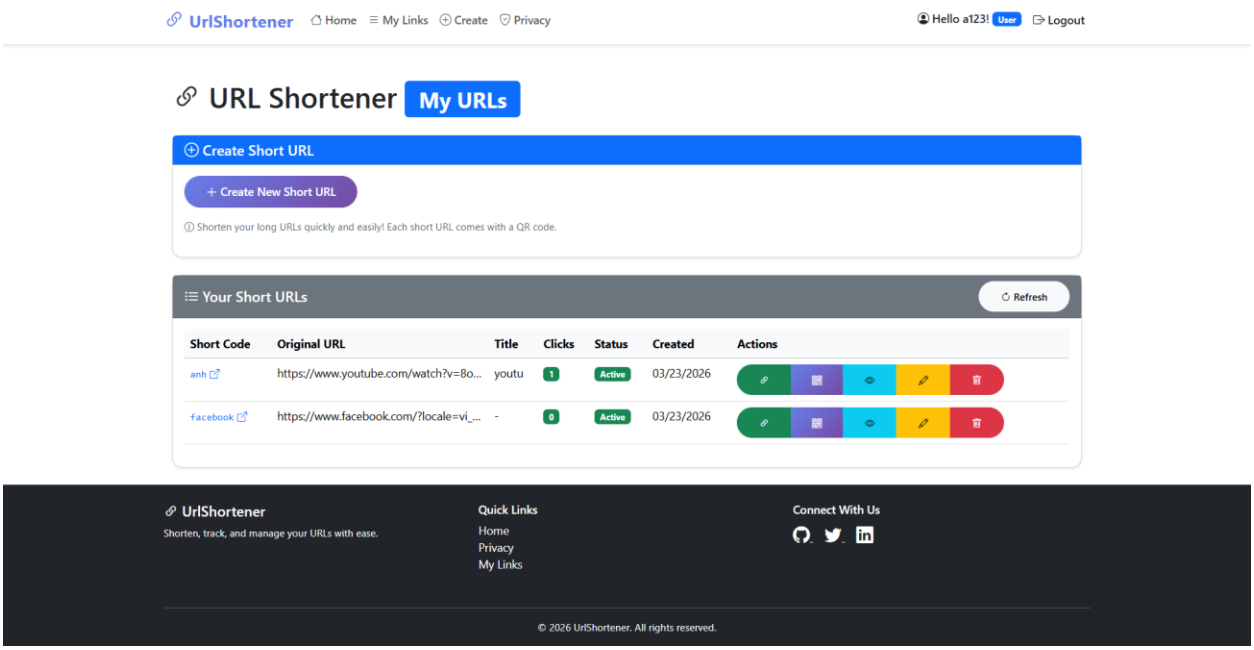

Figure 6. My Links Page

In ***Figure 7***,***Figure 8***, ***Figure 9*** and ***Figure 10*** is the link management system is optimized through specialized functional modules, forming a tightly controlled data management cycle. The QR Code interface in ***Figure 7*** serves as a physical decoding mechanism, enabling mobile devices to quickly connect and instantly access the original source via camera scanning. This logical flow continues in the Detail page in ***Figure 8***, where the system ensures data transparency, allowing users to accurately monitor interaction performance. To maintain flexibility, the Edit function in ***Figure 9*** enables users to actively modify parameters according to their preferences and practical needs without altering the original identifier. Finally, the Delete function in ***Figure 10*** acts as a system filter, allowing users to remove links that are no longer needed, thereby ensuring efficiency and maintaining the security of personal digital resources.

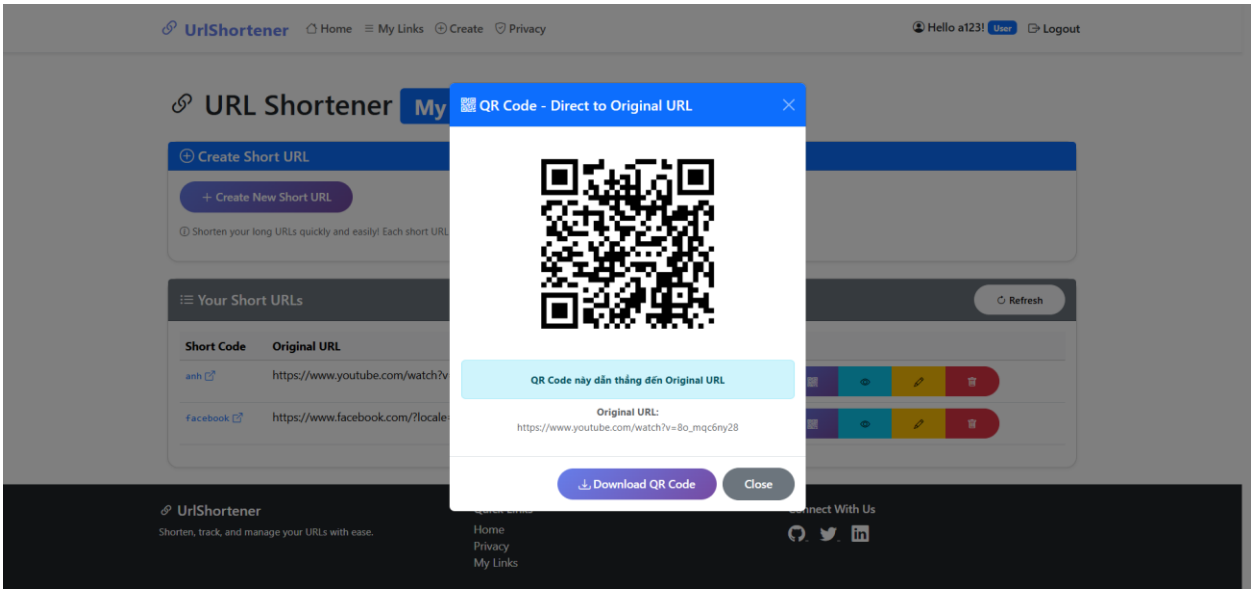

Figure 7. QR Code

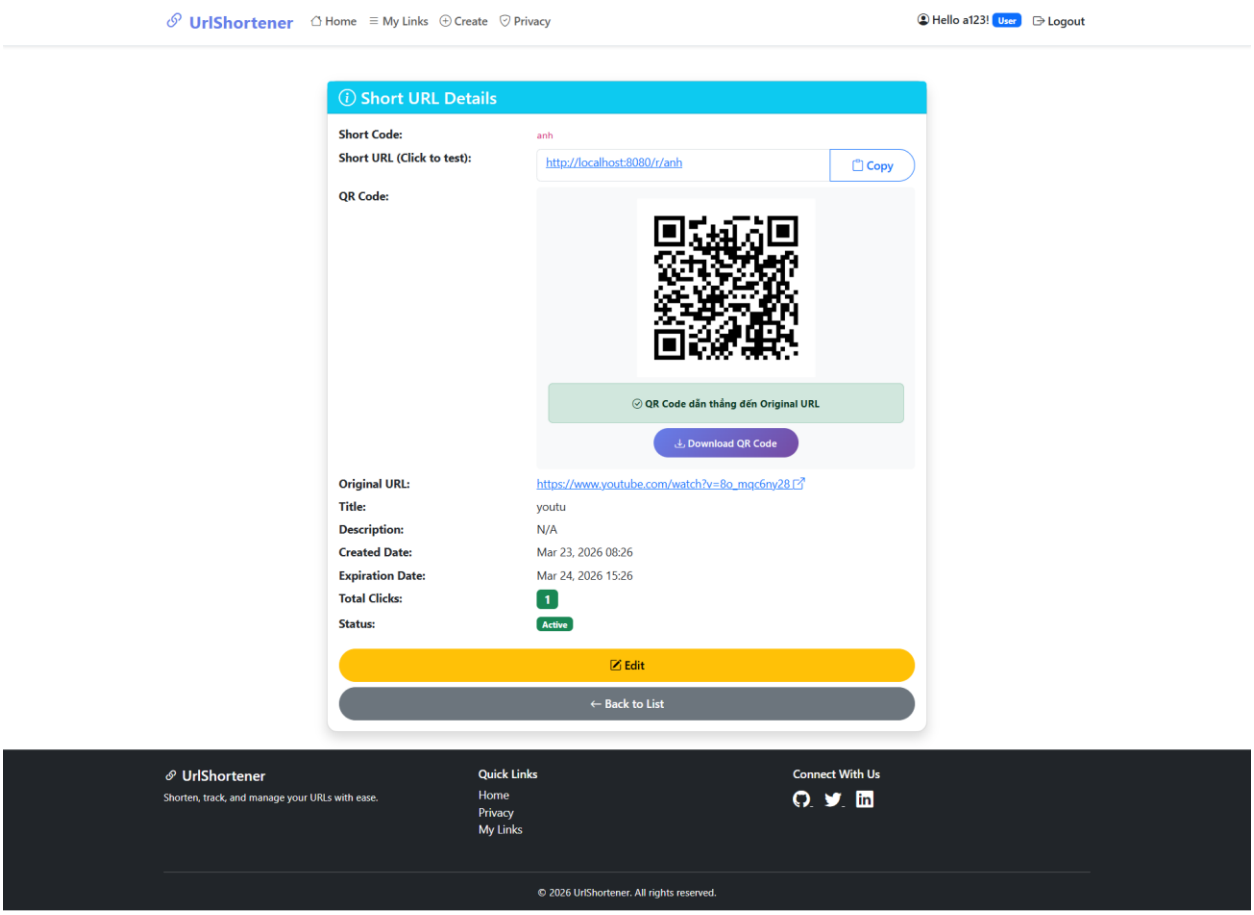

Figure 8. Short URL Detail Page

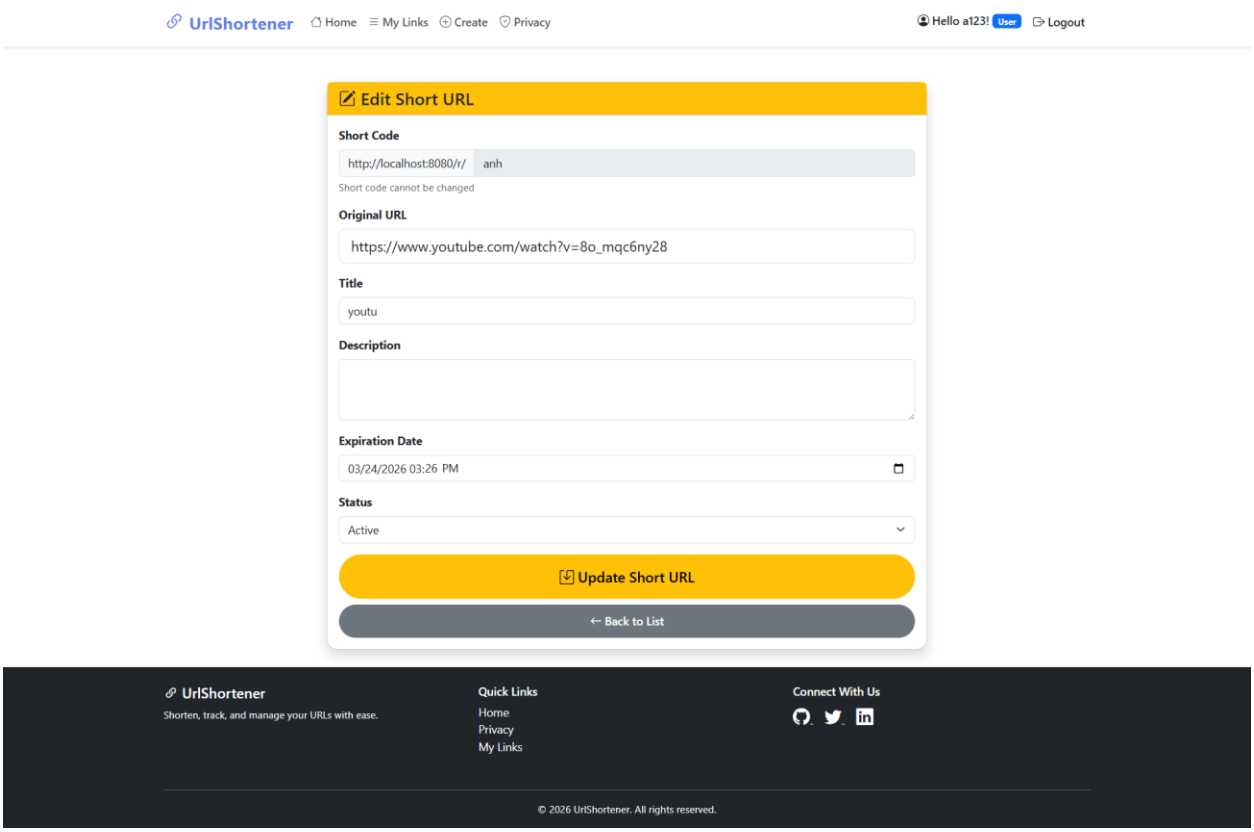

Figure 9. Edit Short URL Page

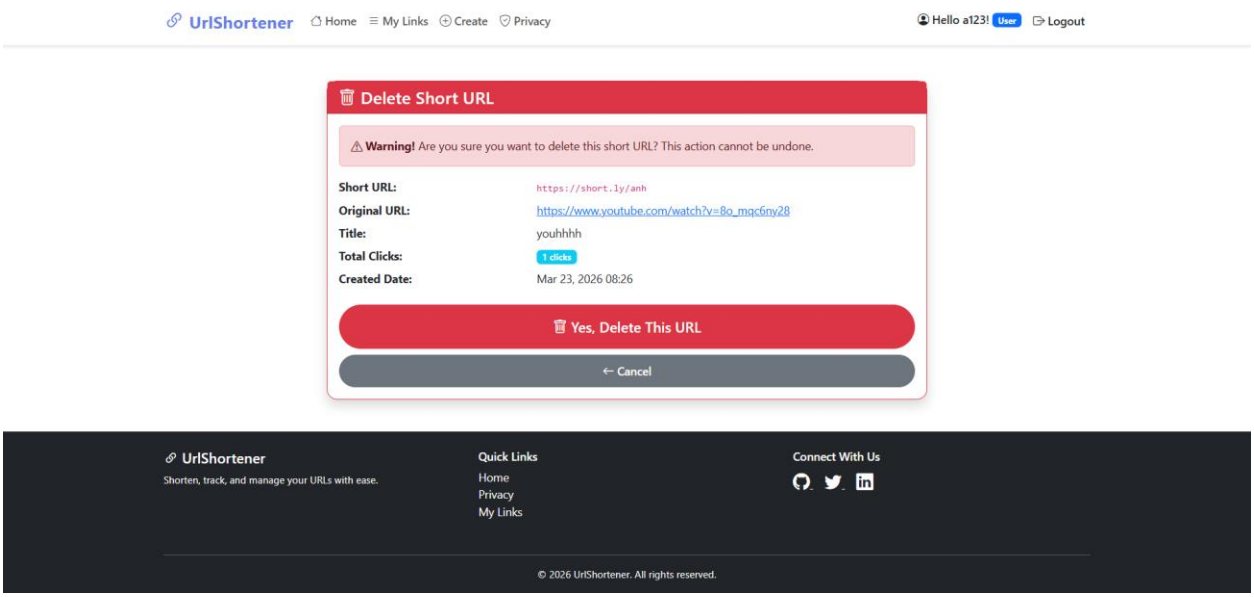

Figure 10. Detele Short URL Page

In ***Figure 11***, the Admin page maintains consistency with the user interface while extending the scope of control across the entire database. The core distinction lies in its comprehensive monitoring capability: instead of displaying only personal links, the system enables administrators to access and view detailed information for all URLs generated by every user within the system. This logical execution is reinforced through full CRUD permissions, allowing administrators not only to monitor but also to directly intervene by editing or deleting any links that violate policies or are no longer appropriate. This integration ensures a centralized management process, supporting transparency and safeguarding the overall integrity of the URL shortening ecosystem.

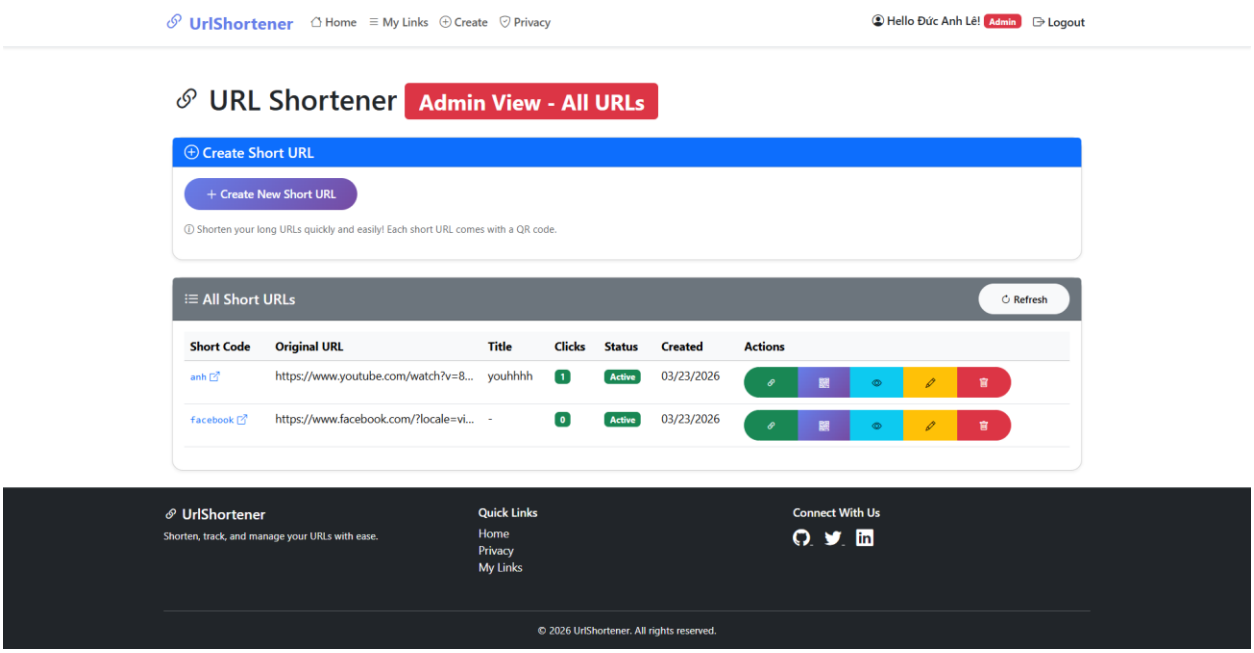

Figure 11. Admin Page

In ***Figure 11***, ***Figure 12*** , the two images show how the system displays warnings and prompts when the user enters incorrect information into the system.

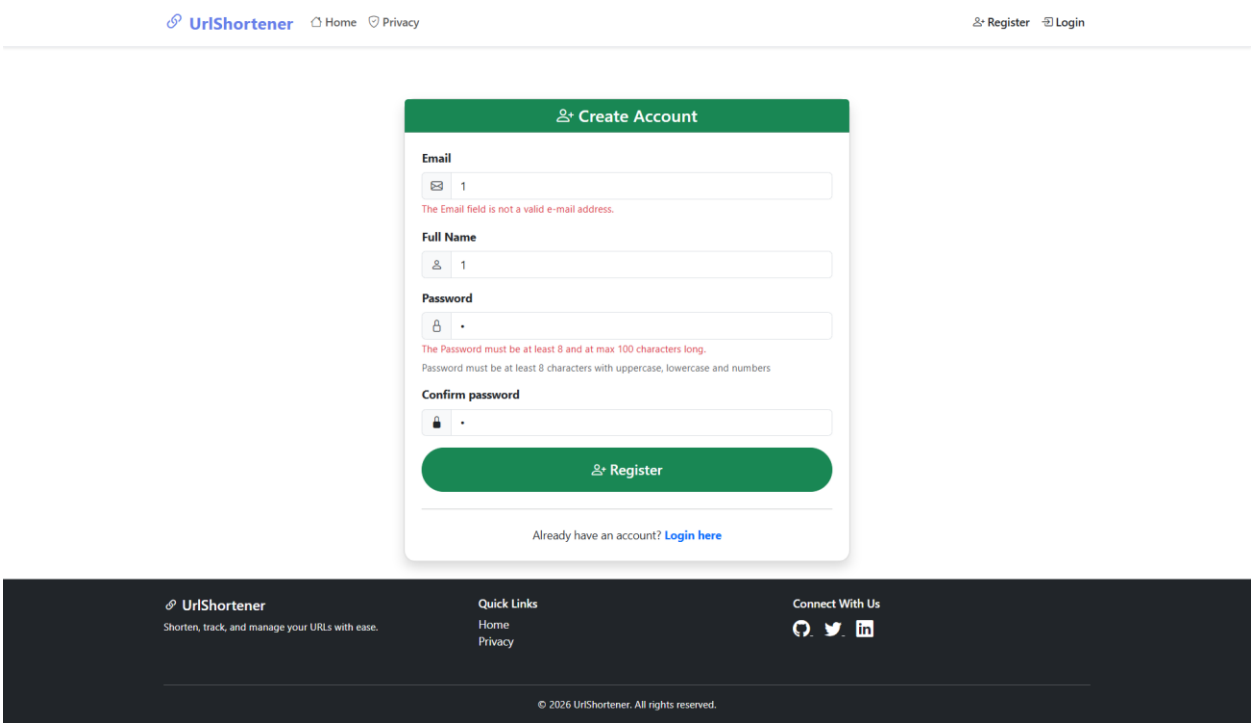

Figure 12. Incorrect information entered

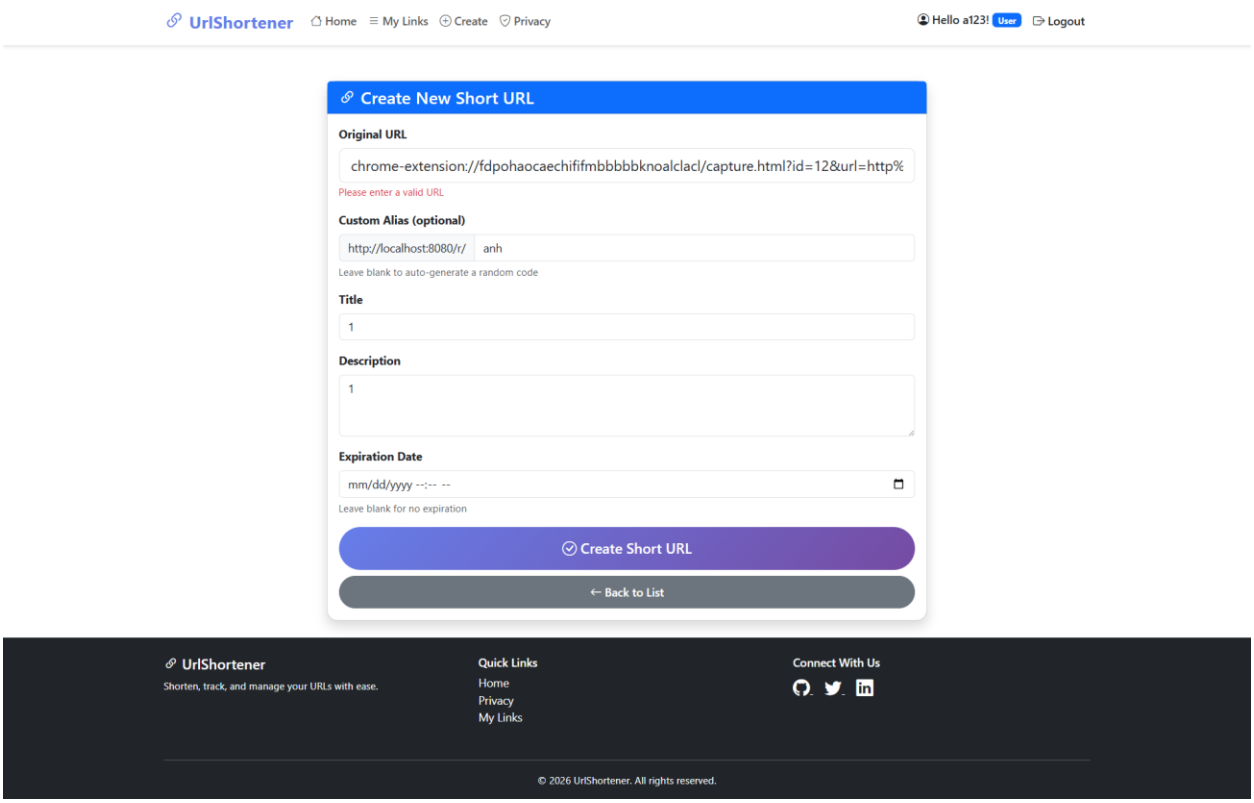

Figure 13. Incorrect URL entered

2.3 Microsoft SQL database

The project utilizes Microsoft SQL Server 2022 as a centralized data storage platform, ensuring system integrity and security. Through the dedicated SQL Server Management Studio (SSMS) 2022 administration tool, data entities ranging from user accounts to linked categories are systematically organized into relational tables.

Below are images showing the system data when a user interacts with the system, as shown in ***Figure 14*** and ***Figure 15***.

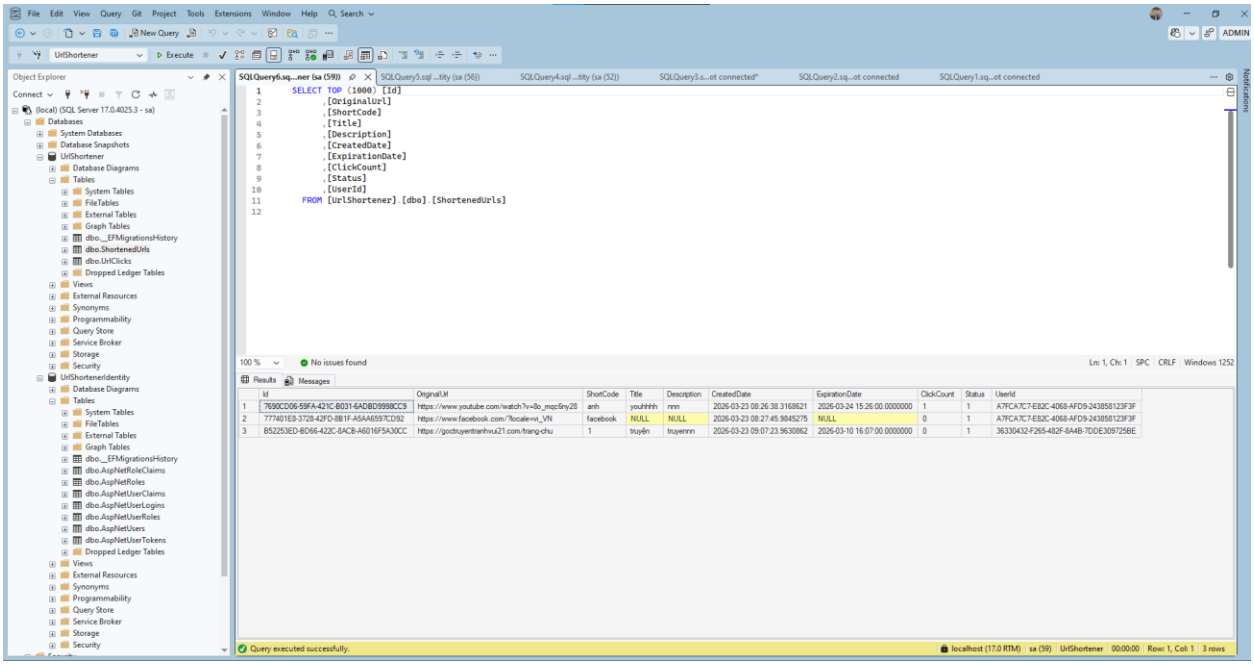

Figure 14. Data of URL links in SQL

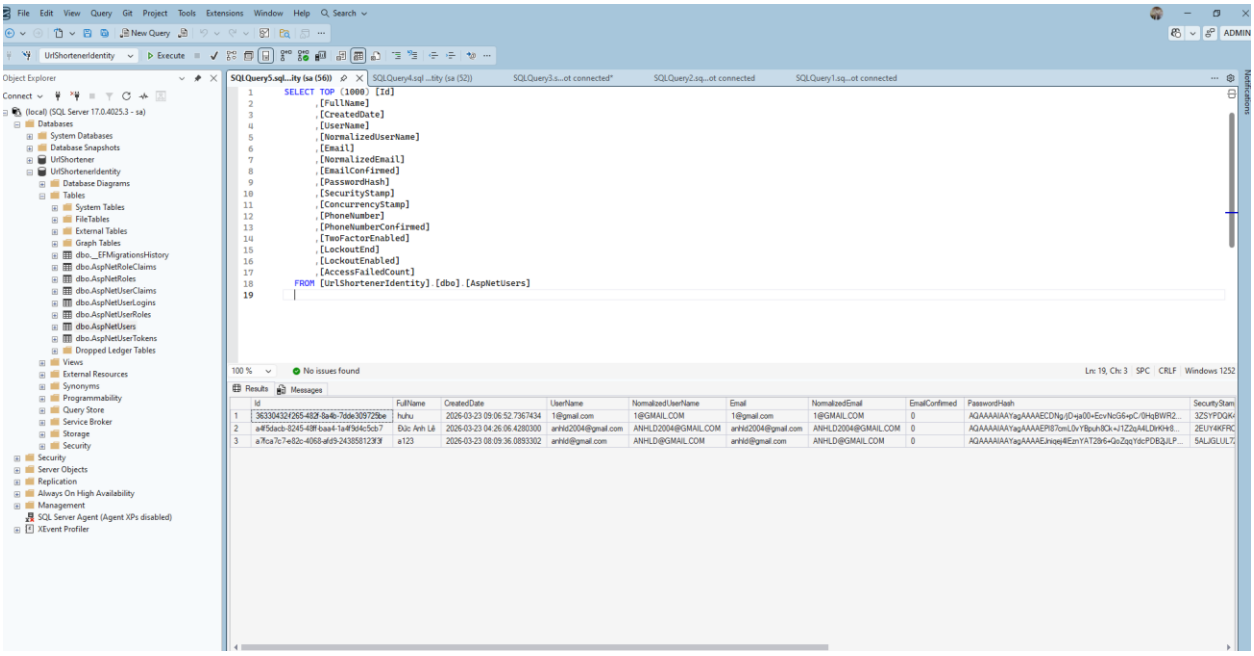

Figure 15. Data of Users in System

3.Conclusion

In conclusion, this project demonstrates the effective application of ASP.NET MVC in developing a functional and user-friendly URL shortening system. By integrating backend logic, database management, and an interactive user interface within a unified architecture, the system ensures both usability and maintainability. The project also provides a solid foundation for future enhancements, including features such as advanced analytics, rate limiting, and link expiration to improve system control and prevent misuse. These potential improvements would further enhance the system's reliability, scalability, and overall value in real-world applications.

4. Self-evaluation

This section discusses the contributions, challenges, and lessons learned from the project.

4.1 Contribution

The development of the URL Shortener system was a collaborative effort among three team members, with responsibilities distributed based on each member's role and expertise. The workload was balanced equally, with each member contributing approximately 33.3% to the overall project.

1. Lê Đức Anh – Team Leader, Tester, and Report Writer (33.3%)

As the team leader, Đức Anh was responsible for project coordination, quality assurance, and documentation. Key contributions include:

+ **Project Management & System Design Oversight**: Coordinated team activities, ensured alignment with project requirements, and reviewed system architecture to maintain consistency and scalability.

+ **Testing & Quality Assurance:** Conducted comprehensive testing of core functionalities including URL shortening, redirection, authentication, and analytics tracking. Identified bugs and ensured system reliability through iterative testing.

+ **Report Writing & Documentation:** Authored the full technical report, including system design, implementation details, and evaluation. Ensured academic writing standards, logical structure, and clarity of explanation.

2. Ông Phước Đại – UI/Frontend Development & Core Functionalities (33.4%)

Phước Đại was primarily responsible for implementing the user interface and a significant portion of the application's core features:

+ **Frontend Development (ASP.NET Core MVC):** Designed and implemented responsive UI components using Razor Views, ensuring user-friendly interaction and clear navigation.

+ **Core Functionalities Implementation:** Developed the URL shortening service (ShortenedUrlService) with collision-free short code generation using encoding.Implemented retry logic to guarantee uniqueness via EF Core queries.Built CRUD operations for shortened URLs, including user-specific filtering and admin views.

+ **Additional Features:** Integrated QR code generation and download functionality using the QRCoder library.

3. Đặng Quốc Khang – Docker & Database Management (33.3%)

Quốc Khang focused on backend infrastructure, database design, and deployment support:

+ **Database Design & Implementation:**Created EF Core entities (ShortenedUrl, UrlClick) and configured relationships using Fluent API. Implemented indexing strategies, including a unique index on ShortCode for performance and data integrity. Managed migrations and schema initialization.

+ **Click Tracking & Analytics System:** Developed the UrlClickService to capture request data such as IP address, timestamp, and geolocation hints, along with aggregated click statistics.

+ **Infrastructure & Deployment:** Configured Docker environment using docker-compose.yml for containerized deployment.Integrated Redis caching for performance optimization. Supported ASP.NET Identity setup and application configuration in Program.cs.

4.2 Challenges

During development, several challenges were encountered. Ensuring the uniqueness of shortcodes in concurrent requests proved difficult, as the current approach could lead to large-scale data conflicts. Managing EF Core relationships, particularly sequential deletion between ShortenedUrl and UrlClick, required careful configuration, while IP geolocation functionality was still incomplete. Integrating ASP.NET Identity with a separate DbContext increased system complexity, and the allocation of administrator users was currently handled manually. The implemented global rate limiting lacked granularity, as it did not differentiate between users or resources. Additionally, setting up Docker and Redis presented configuration challenges, especially in maintaining stable connections. Due to time constraints, no unit or integration testing was performed. Ultimately, security remains limited, with basic authentication and minimal input verification, and no mandatory HTTPS or advanced protection mechanisms.

4.3 Learning Outcomes

Đức Anh – Team Leader, Tester, and Report Writer:

Throughout the project, Đức Anh developed a stronger understanding of project coordination and software quality assurance. Dealing with challenges such as concurrency issues, limited security implementation, and the absence of automated testing emphasized the importance of planning, risk awareness, and systematic validation in software development. In addition, documenting these limitations helped improve his ability to critically assess system design and present technical ideas in a clear, structured, and academically appropriate manner.

Phước Đại – UI/Frontend Development & Core Functionalities:

Phước Đại gained practical experience in implementing both frontend and core backend functionalities within the ASP.NET Core MVC framework. Working on features such as shortcode generation and rate limiting helped him better understand how to handle concurrent requests and design scalable systems. At the same time, dealing with EF Core relationships and partially implemented features like IP geolocation allowed him to recognize the importance of data consistency and the role of external services in completing real-world applications.

Quốc Khang – Docker & Database Management:

Quốc Khang strengthened his knowledge in database design and system infrastructure through hands-on experience with EF Core and deployment tools. Managing entity relationships and integrating ASP.NET Identity with a separate DbContext provided deeper insight into the complexity of backend architecture. Additionally, resolving issues related to Docker and Redis configuration helped him better understand containerized environments and the importance of stable system integration in distributed applications.

Student Assessment Submission and Declaration

When submitting evidence for assessment, each student must sign a declaration confirming that the work is their own.

<table>
<tr><td colspan="2">Student name:
1. Lê Đức Anh
2. Ông Phước Đại
3. Đặng Quốc Khang</td><td colspan="2">Assessor name:
CanhHT5</td></tr>
<tr><td>Issue date:</td><td>Submission date:
24/04/2026</td><td colspan="2">Submitted on:
24/03/2026</td></tr>
<tr><td colspan="4">Programme:
Advanced Microservices Development and Deployment</td></tr>
<tr><td colspan="4">Unit: AMD201</td></tr>
<tr><td colspan="4">Assignment number and title: Group 3 and Short URL System</td></tr>
</table>

Plagiarism

Plagiarism is a particular form of cheating. Plagiarism must be avoided at all costs and students who break the rules, however innocently, may be penalised. It is your responsibility to ensure that you understand correct referencing practices. As a university level student, you are expected to use appropriate references throughout and keep carefully detailed notes of all your sources of materials for material you have used in your work, including any material downloaded from the Internet. Please consult the relevant unit lecturer or your course tutor if you need any further advice.

Student Declaration

Student declaration
I certify that the assignment submission is entirely my own work and I fully understand the consequences of plagiarism. I declare that the work submitted for assessment has been carried out without assistance other than that which is acceptable according to the rules of the specification. I certify I have clearly referenced any sources and any artificial intelligence (AI) tools used in the work. I understand that making a false declaration is a form of malpractice.

Student signature: Anh, Đại, Khang Date: 24/03/2026